Лекция 5

Целевая архитектура и управление переходом



Варианты, GAP-анализ, проверка и оценка эффекта

Результаты изучения темы

Нужно научиться обосновывать целевое состояние, объяснять переход и различать предположенный эффект, наблюдение прототипа и подтверждённый результат внедрения.

Проектировать

Сравнивать варианты и описывать TO-BE.

Планировать

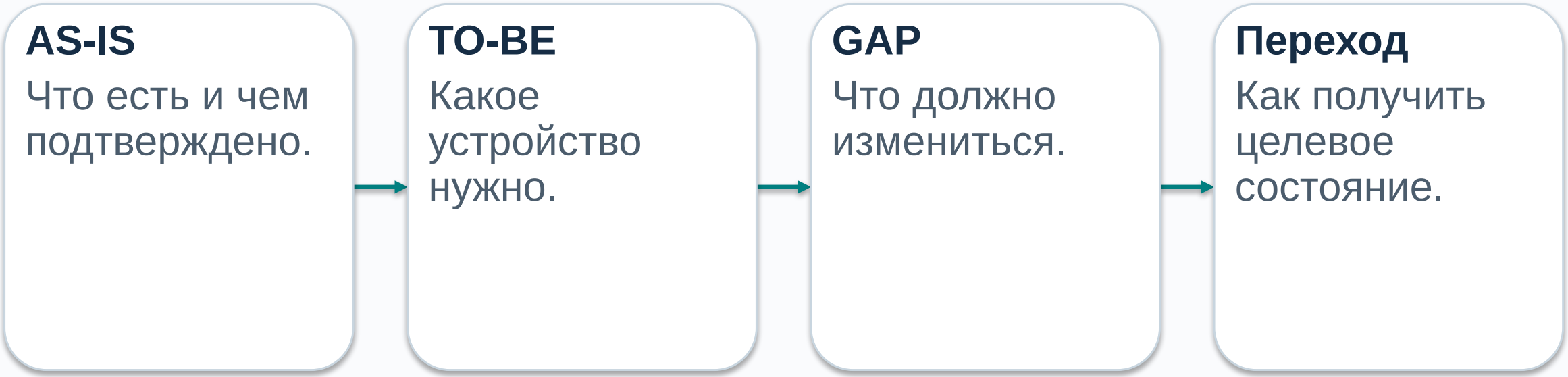
Связывать разрывы, зависимости и инициативы.

Оценивать

Проверять архитектуру, риски и механизм эффекта.

Логика архитектурного изменения

Описание состояний, анализ разрывов и план перехода связаны. При появлении новых сведений возможен возврат к ранее принятому решению.



AS-IS — не перечень недостатков

Описание текущего состояния включает и работающие части процесса. Без них невозможно оценить, что нужно сохранить при изменении.

Сохраняемое

Функции и данные, которые уже обеспечивают результат.

Проблемное

Подтверждённые ограничения и затруднения.

Неизвестное

Вопросы, для которых пока недостаточно оснований.

Базовое состояние для сравнения

Базовое состояние фиксирует исходные процессы, показатели и условия. Изменение сравнивают с ним по одинаковым правилам.

Граница

Тот же процесс и набор существенных объектов.

Период

Известный момент или интервал наблюдения.

Показатели

Согласованные единицы и способ измерения.

Нельзя доказать улучшение, незаметно изменив условия сравнения.

Требования направляют проектирование

Требования определяют необходимые свойства целевой деятельности и ИС. Каждое существенное требование должно иметь основание и способ проверки.

Основание

Почему это свойство необходимо.

Область

К какой функции или объекту оно относится.

Проверка

Как установить, что результат достаточен.

TO-BE: достижимое целевое состояние

Целевая архитектура описывает устройство, обеспечивающее согласованные требования в реальных ограничениях. Она не обязана устранять все проблемы предприятия сразу.

Устройство

Процессы, ответственность, данные, приложения и технологии.

Обоснование

Связь компонентов с требованиями.

Ограничение

Явное описание того, что остаётся за рамками.

Принципы ограничивают произвольность выбора

Принцип должен объяснять, как принимать решения в повторяющихся ситуациях. Его последствия применяются ко всем затронутым представлениям.

Принцип

Одна хозяйственная операция имеет устойчивый идентификатор.

→

Следствие

Код сохраняется при передаче и повторе.

→

Проверка

Операция прослеживается в реестре и отчёте.

Ограничения нужно проверять до сравнения баллов

Вариант, нарушающий обязательное условие, нельзя «компенсировать» высокой оценкой удобства. Сначала проверяют допустимость, затем предпочтительность.

Обязательное условие

Не менять действующую учётную систему.

Недопустимый вариант

Требует её полной замены.

Допустимые варианты

Регламент, управляемый обмен или дополнительный модуль.

Формирование альтернатив

Для одной проблемы обычно существуют разные типы решений. Важно рассмотреть организационный вариант наряду с программным и инфраструктурным.

Организационный

Упорядочить роли, сроки и правила.

Интеграционный

Связать существующие системы.

Программный

Изменить или создать необходимый компонент.

Вариант без нового внедрения

Сохранение текущего устройства — тоже вариант сравнения. Он помогает увидеть цену изменения и цену отказа от него.

Сохраняется

Знакомый процесс и действующие затраты.

Остаётся

Подтверждённая проблема и связанные риски.

Вопрос

Оправдывает ли ожидаемый результат затраты перехода?

Сравнение вариантов по одинаковым признакам

Критерии выбирают до оценки. Для каждого варианта проверяют одинаковую область результата.

Критерий	Что выясняем	Основание
Требования	Что выполняется	Модель и проверка
Переход	Что потребуется изменить	GAP и зависимости
Сопровождение	Кто сможет поддерживать	Роли и ресурсы

Фильтр допустимости

Фильтр отделяет обязательные условия от предпочтений. Неизвестное выполнение критичного условия требует проверки, а не автоматического высокого балла.

Допустим

Выполнение обязательных условий подтверждено.

Неизвестен

Необходим эксперимент или уточнение.

Недопустим

Есть нарушение обязательного ограничения.

Компромиссы должны быть видимыми

Выбор может улучшать одно свойство и ухудшать другое. Архитектор объясняет, почему конкретный компромисс приемлем в данной ситуации.

Выгода

Автономный ввод снижает зависимость от сети.

Цена

Нужно согласовывать локальные и центральные состояния.

Условие

Назначен владелец конфликтов и проверены сценарии обмена.

Учебная модель многокритериальной оценки

После фильтра допустимости сравним варианты по шкале 1–5: больше — предпочтительнее. Веса выражают выбранные приоритеты, а не объективную истину.

Критерий	Вес	Регламент	Обмен
Соответствие задаче	0,5	3	5
Простота перехода	0,3	5	3
Сопровождаемость	0,2	4	4

Сводная оценка одного варианта

У варианта управляемого обмена оценки равны 5, 3 и 4. Взвешенная сумма позволяет прозрачно показать влияние выбранных приоритетов.

$$\text{Оценка} = 0{,}5 \times 5 + 0{,}3 \times 3 + 0{,}2 \times 4 = 4{,}2$$

Оценки	5 — соответствие; 3 — переход; 4 — сопровождение.
Сравнение	Для остальных вариантов применяется та же шкала.
Ограничение	Число не заменяет проверку требований и рисков.

Чувствительность оценки к приоритетам

Победитель может измениться при другом наборе весов. Поэтому близкие оценки нужно проверять сценариями и обсуждать с заинтересованными сторонами.

Исходные веса

Регламент: 3,8; управляемый обмен: 4,2.

Переход важнее

При весах 0,3 / 0,5 / 0,2 получаем 4,2 и 3,8.

Вывод

Выбор зависит от приоритетов, а не от магии одной суммы.

Обоснованная запись решения

Итог сравнения сохраняется вместе с отвергнутыми альтернативами и условиями пересмотра. Тогда решение можно объяснить другому участнику.

Выбрано

Управляемый обмен существующих систем.

Почему

Поддерживает требования без полной замены.

Цена и пересмотр

Нужны сопоставление данных, владелец обмена и проверка отказов.

Целевая бизнес-архитектура

Целевая бизнес-модель должна показать изменение деятельности и ответственности. Новый сервис не заменяет правила использования его результата.

Процесс

Как проходит работа после изменения.

Роли

Кто создаёт, проверяет и принимает результат.

Исключения

Кто действует при неполных или спорных сведениях.

Целевая архитектура данных

На целевом уровне определяют смысл объектов, источники, правила изменения и допустимые копии. Особенно важно сохранить соответствие старых и новых идентификаторов.

Справочники

Устойчивые коды и единицы.

Операции

Идентичность, статус и происхождение.

Ответственность

Владелец и правила исправления.

Целевая архитектура приложений

Выбираются обязанности приложений и их взаимодействия. Новая функция может потребовать изменения существующего приложения, а не обязательно создания нового.

Сохранить

Использовать пригодные функции действующих систем.

Дополнить

Закрыть конкретный функциональный разрыв.

Устранить дубли

Прекратить лишнюю независимую обработку.

Целевая технологическая архитектура

Техническая среда должна поддерживать необходимые сценарии нагрузки, связи, защиты и восстановления. Выбор технологии объясняется этими условиями.

Размещение

Где работают компоненты.

Зависимости

Какие ресурсы и каналы им необходимы.

Эксплуатация

Как наблюдается и восстанавливается работа.

Безопасность целевого состояния

Целевая модель должна определять допустимые действия, границы доверия и обработку нарушений. Безопасность нельзя отложить до завершения внедрения.

Полномочия

Кто работает с каким объектом.

Проверки

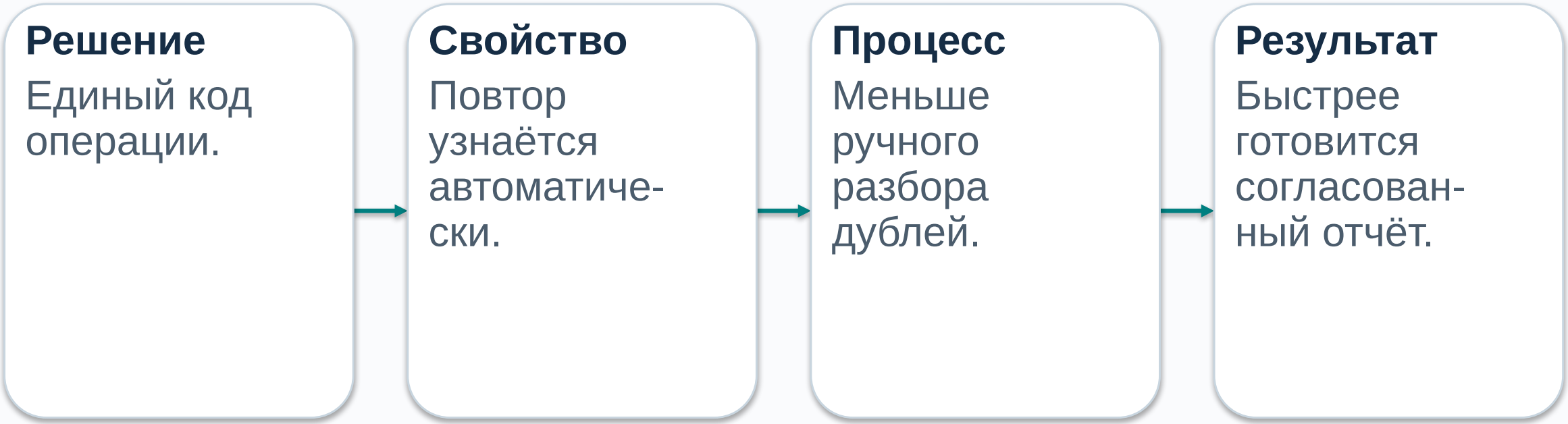
Какие данные считаются принятыми.

Контроль

Как обнаружить отклонение и восстановить состояние.

Изменение должно иметь механизм эффекта

Полезно связать техническое решение с качеством системы, затем — с изменением процесса и результатом предприятия.



Трассируемость целевой модели

Для каждого значимого решения проверяют связь с требованием, проблемой и критерием. Для каждого требования — наличие поддерживающего решения.

Снизу вверх

Что обосновывает компонент или правило?

Сверху вниз

Чем реализовано требуемое свойство?

Проверка

Как будет подтверждён ожидаемый результат?

Понятие GAP-анализа

GAP-анализ устанавливает различия между текущим и целевым состояниями в согласованных границах. Разрыв может относиться к процессу, данным, системе, роли или качеству.

AS-IS

Как устроено сейчас.

→

TO-BE

Что требуется в целевом состоянии.

→

Разрыв

Какое изменение необходимо.

Характер архитектурного изменения

Разрыв не всегда означает покупку или разработку нового компонента. Часть элементов сохраняется, меняет роль, объединяется или прекращает работу.

Сохранить

Оставить пригодный элемент.

Изменить

Уточнить роль или поведение.

Добавить

Создать недостающую возможность.

Прекратить

Убрать лишний или заменённый механизм.

Структура таблицы GAP

Строка должна связывать подтверждённое состояние и необходимое изменение.

Сейчас	Требуется	Изменение
Разные коды	Единая идентичность	Сопоставить справочники
Неясный статус	Явное подтверждение	Назначить правило и роль
Повторный ввод	Один бизнес-факт	Контролировать повтор

Учебный пример

Бизнес-разрыв

Бизнес-разрыв возникает, когда текущая организация работы не обеспечивает требуемую способность или результат.

AS-IS

Подтверждение факта выполняется нерегулярно.

TO-BE

Каждая значимая операция имеет ответственного и статус.

Изменение

Согласовать порядок подтверждения и разбора исключений.

Учебный пример

Разрыв в данных

Разрыв в данных может быть связан с отсутствием объекта, неясной идентичностью, качеством или несогласованным жизненным циклом.

AS-IS

Один корм записан под разными названиями.

TO-BE

Существует управляемый справочник кодов.

Изменение

Назначить владельца, сопоставить записи и определить правила изменения.

Учебный пример

Разрыв интеграции

Разрыв может находиться не внутри приложения, а в передаче результата между системами.

AS-IS

Файл повторяют без понятного статуса.

TO-BE

Отправитель знает, принят ли конкретный пакет.

Изменение

Ввести идентификатор, подтверждение и разбор ошибок.

Технологический разрыв

Технологический разрыв связан с невозможностью обеспечить требуемые условия работы при текущей инфраструктуре.

AS-IS

Потеря связи останавливает регистрацию.

TO-BE

Допустима локальная фиксация с последующим согласованием.

Изменение

Обосновать локальное хранение и надёжность обмена.

Разрыв компетенций и сопровождения

Даже технически подходящее решение может быть неустойчивым, если его некому поддерживать. Компетенции и ответственность входят в архитектурные ограничения.

Новая обязанность

Контролировать очередь ошибок обмена.

Необходимый ресурс

Исполнитель, инструкция и доступ.

Проверка

Другой участник способен диагностировать и обработать типичную ошибку.

Разрыв, требование и задача проекта

Требование задаёт нужное свойство. Разрыв показывает различие состояний. Задача проекта — действие по реализации изменения.

Требование

Не учитывать повтор операции.

→

Разрыв

AS-IS не распознаёт повтор.

→

Задача

Добавить механизм идентификации и проверить его.

Одна инициатива может закрывать несколько разрывов

Сопоставление справочника требует не только программного преобразования, но и решений о владельце, кодах, правилах изменения и обмене.

Организация
Назначить владельца.

→

Данные
Установить соответствие кодов.

→

Приложения
Применять согласован-ные значения.

→

Контроль
Проверять изменения и ошибки.

Не нужно автоматически превращать каждую строку GAP в отдельный ИТ-проект.

Учебный пример

Зависимости определяют порядок перехода

Работа может начаться технически, но не дать полезного результата до готовности зависимого условия. Дорожная карта должна отражать такие отношения.

Справочник

Согласовать смысл и идентификаторы.

→

Обмен

Передавать сопоставленные операции.

→

Отчёт

Использовать подтверждённые сведения.

Что можно делать параллельно

Параллельность определяется независимостью результатов, а не желанием ускорить все работы сразу. Общие интерфейсы и данные требуют предварительного согласования.

Параллельно

Готовить интерфейс и тестовые данные после согласования контракта.

Последовательно

Нельзя окончательно проверить обмен до определения смысла полей.

Переходная архитектура

Переходное состояние описывает временное устройство между AS-IS и TO-BE. Оно должно быть работоспособным и иметь понятные условия завершения.

Сосуществование

Старый отчёт и новый контур сверяются.

Временные правила

Определён авторитетный источник на каждом шаге.

Завершение

Известно, когда прекращается прежний маршрут.

Учебный пример

Временный двойной ввод требует контроля

Иногда двойной ввод допустим на этапе проверки. Без срока и критерия прекращения он может стать постоянной дополнительной нагрузкой.

Цель

Сравнить новый результат со старым.

Ограничение

Одна площадка и определённый период.

Выход

Согласованный критерий отказа от старого ввода.

Пилотное внедрение

Пилот проверяет решение на ограниченном участке деятельности. Он позволяет наблюдать не только программу, но и работу пользователей и организационных правил.

Граница

Площадка, процесс и участники.

Критерии

Показатели и существенные сценарии.

Решение

Условия расширения, изменения или остановки.

Критерии объявляются до результата

До пилота нужно определить, что будет считаться успехом. Иначе возникает соблазн назвать удачным любой наблюдаемый результат.

Функция

Значимые операции выполняются.

Качество

Ошибки и задержки укладываются в согласованные условия.

Эксплуатация

Есть владелец, поддержка и восстановление.

Граница вывода по пилоту

Успех на одной площадке относится к её условиям. Другая нагрузка, связь, данные и компетенции могут изменить результат.

Подтверждено

Решение прошло заданные проверки на пилоте.

Не подтверждено

Одинаковое поведение во всех других условиях.

Следующий шаг

Выделить отличия площадок и дополнительные проверки.

Неудачная проверка может быть полезной

Корректно зафиксированный отрицательный результат снимает неопределённость и помогает пересмотреть решение. Он не делает исследование бессмысленным.

Наблюдение

После разрыва связи возникает конфликт версий.

→

Вывод

Прежнее правило синхронизации недостаточно.

→

Изменение

Уточнить состояния, владельца конфликта и проверку.

Прототип — средство проверки гипотезы

Прототип создаётся для проверки конкретного предположения об устройстве или поведении. Он не обязан быть полноценным продуктом.

Вопрос

Можно ли распознать повтор одной операции?

Механизм

Минимальная реализация необходимой обработки.

Результат

Наблюдение, поддерживающее или опровергающее гипотезу.

Типы учебного прототипа

Форма прототипа определяется проверяемым вопросом. Разные прототипы подтверждают разные свойства.

Макет интерфейса

Проверяет понимание и последовательность действий.

Модель обработки

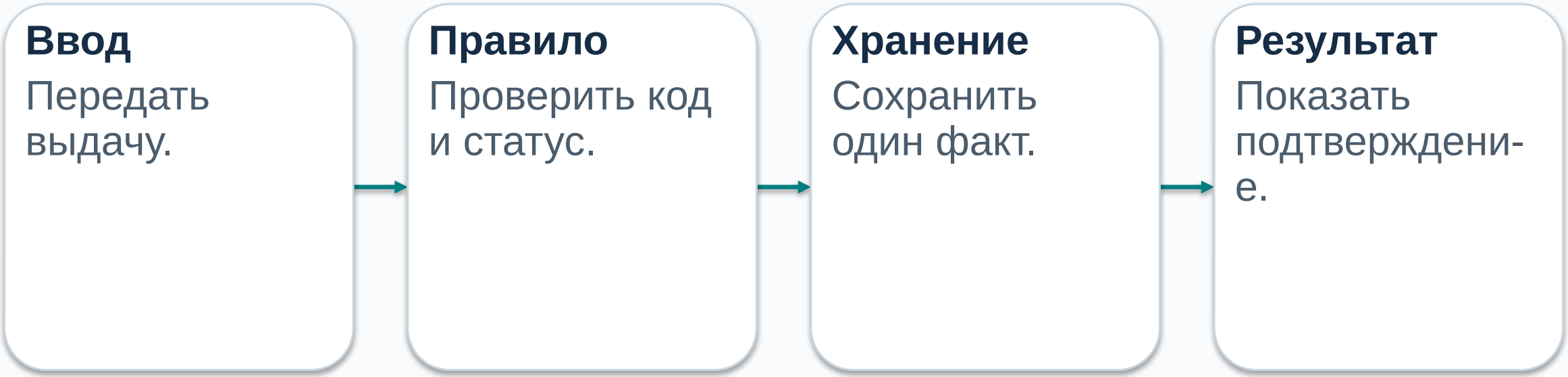
Проверяет правило или алгоритм на заданных данных.

Интеграционный фрагмент

Проверяет конкретное взаимодействие компонентов.

Вертикальный срез решения

Вертикальный срез связывает один пользовательский сценарий через несколько слоёв: ввод, правило, хранение и результат. Он полезнее набора несвязанных экранов.



Паспорт прототипа

Паспорт фиксирует вопрос, входы, среду, реализованные механизмы, ожидаемые наблюдения и ограничения. Имитация должна быть отделена от реально работающей части.

Реализовано

Проверка повторного идентификатора.

Имитируется

Ответ внешней учётной системы.

Вывод ограничен

Реальная интеграция с этой системой ещё не проверена.

Тестовые данные должны иметь назначение

Набор примеров создаётся под проверяемые свойства. Нужно знать ожидаемый результат и отдельно сохранить исходные данные.

Обычные

Корректная операция.

Граничные

Пустое поле или предельное значение.

Проблемные

Повтор, конфликт и недоступная зависимость.

Синтетические данные помечаются явно и не выдаются за выборку реального предприятия.

Учебный пример

Нормальный сценарий проверки

Нормальный сценарий показывает ожидаемое выполнение при допустимых данных и доступных зависимостях.

Вход

Новая операция с допустимыми полями.

Ожидание

Создана одна запись и возвращён понятный статус.

Наблюдение

Проверены и интерфейс, и сохранённые данные.

Учебный пример

Граничный сценарий проверки

Граничные случаи помогают обнаружить скрытые предположения алгоритма. Нужны не случайные ошибки, а условия, связанные с требованиями.

Случай

Количество не указано.

Ожидание

Операция не подтверждается.

Результат для человека

Пользователь получает понятное указание, что исправить.

Учебный пример

Сценарий отказа

Проверка отказа рассматривает нарушение конкретной зависимости и сохранение допустимого состояния системы.

Нарушение

Ответ сервера потерян.

Ожидание

Операция не объявляется безусловно неисполненной.

Проверка

Повтор или уточнение статуса не создаёт второй расход.

Воспроизводимость проверки

Воспроизводимость означает, что другой участник может повторить существенные условия и получить сопоставимое наблюдение.

Входы

Сохранены данные и ожидаемые результаты.

Среда

Известны версия и способ запуска.

Наблюдение

Зафиксированы состояние и границы вывода.

Регрессионная проверка после изменения

Исправление одного случая может нарушить другой. После значимого изменения повторяют ранее важные проверки и сравнивают результаты.

До изменения

Сохранён набор пройденных сценариев.

→

Изменение

Уточнено правило обработки.

→

После

Проверено, что требуемое поведение сохранилось.

Демонстрация и доказательство свойства

Показ работающего экрана доказывает только наблюдавшееся поведение в заданных условиях. Он не подтверждает автоматически безопасность, нагрузку и все отказные сценарии.

Показано

Одна операция успешно сохранена.

Нужно отдельно проверить

Повтор, отказ, права и конфликт.

Честный вывод

Указать, какие свойства проверены, а какие нет.

Архитектурная экспертиза

Architecture Review проверяет согласованность проекта с требованиями и выявляет риски. Предмет обсуждения — основания и последствия решения, а не любимые технологии рецензента.

Основание

Подтверждена ли проблема.

Решение

Связаны ли требования и устройство.

Переход

Можно ли получить целевое состояние предложенным способом.

Участники экспертизы

Разные роли замечают разные классы проблем. Их вопросы должны связываться с общими критериями проекта.

Бизнес

Ценность, изменение работы и ответственность.

Разработка

Реализуемость, контракты и согласованность.

Эксплуатация и защита

Наблюдение, восстановление и допустимые действия.

Проверка полноты

Полнота означает наличие необходимых элементов относительно цели и границы. Она не измеряется количеством страниц или диаграмм.

Контекст

Есть задача, участники и ограничения.

Архитектура

Показаны значимые зависимости.

Проверка

Существенные требования имеют критерии.

Проверка согласованности

Согласованность означает отсутствие необъяснённых противоречий между моделями, терминами и решениями.

Объекты

Одинаковые идентификаторы и значения.

Роли

Права совпадают с ответственностью.

Сценарии

Возможное поведение соответствует схеме компонентов.

Риск и ошибка — разные замечания

Ошибка указывает уже обнаруженное противоречие. Риск описывает возможное нежелательное последствие при определённых условиях.

Ошибка

В двух моделях разные владельцы одной записи.

Риск

При росте нагрузки общая БД может стать узким местом.

Действие

Исправить противоречие или проверить и обработать риск.

Качественная матрица рисков

Учебная матрица сопоставляет вероятность и влияние по категориям. Позиции P1–P3 условны: они показывают обсуждение приоритета, а не денежную оценку риска.

Влияние	Вероятность → Низкая	Средняя	Высокая
Высокое		P2	P1
Среднее			
Низкое		P3	

P1 Потеря канала связи.

P2 Несогласованные коды.

P3 Задержка некритичного отчёта.

Нельзя считать произведение условных рангов достоверной денежной оценкой риска.

Ответственный и способ обработки риска

Для существенного риска определяют владельца решения и действие: снизить, избежать, передать в допустимой форме или принять с основанием.

Сценарий	Действие	Остаточный риск
Недоступность канала останавливает регистрацию.	Проверить автономную запись и восстановление обмена.	Потеря устройства остаётся отдельным ограничением.

Технический долг и временное решение

Временный обход может быть оправдан, если известны его последствия и условия завершения. Без владельца и срока он превращается в неуправляемую зависимость.

Контекст

Почему выбран временный механизм.

Обязательство

Что потребуется исправить позже.

Контроль

Кто и при каком событии пересматривает решение.

Проверка безопасности в экспертизе

Эксперт должен проследить конкретный путь данных и полномочий, а не ограничиться наличием слова «шифрование».

Субъект

Кто инициирует действие.

Граница

Где проверяются данные и права.

Последствие

Что происходит при недопустимом обращении.

Проверка производительности

Производительность оценивают при заданной нагрузке и границе измерения. Время запроса к БД не равно времени полного пользовательского действия.

Нагрузка

Операции, объём данных и одновременность.

Показатель

Время ответа, пропускная способность или длительность пакета.

Ограничение

Условия эксперимента и выявленное узкое место.

Проверка сопровождаемости

Сопровождаемость зависит от понятности устройства, доступных компетенций, наблюдаемости и возможности воспроизвести проблему.

Понимание

Другой участник объясняет обязанности компонентов.

Диагностика

Сигнал связан с причиной и действием.

Изменение

Можно внести правку и повторить важные проверки.

Как закрывать замечание экспертизы

Ответ должен сопоставлять исходное замечание с фактическим изменением. Новая картинка сама по себе не закрывает неопределённость ответственности.

Замечание

Не назначен владелец справочника.

→

Изменение

Роль назначена, полномочия и процедура описаны.

→

Проверка закрытия

Модели и сценарий изменения используют это правило.

Дорожная карта архитектурного перехода

Дорожная карта показывает последовательность изменений, зависимости, результаты и критерии завершения. Это не просто список дат или программных задач.

Инициативы

Относительно целостные изменения.

Зависимости

Какие результаты нужны раньше других.

Контроль

Когда переход к следующему этапу допустим.

Этап и контрольная точка

Этап объединяет работы. Контрольная точка фиксирует проверяемую готовность результата и право продолжать переход.

Этап

Подготовить согласованный справочник.

Контрольная точка

Коды сопоставлены, владелец назначен, ошибки разобраны.

Следующий шаг

Можно проверять обмен по согласованному контракту.

Пакет работ должен иметь выход

Для каждой инициативы определяют входные условия, владельца, результат и проверку. Это позволяет отличить выполненное действие от достигнутой готовности.

Вход

Согласованные правила и необходимые данные.

→

Работа

Реализация выбранного изменения.

→

Выход

Подтверждённый результат, пригодный для следующего шага.

Архитектурная зависимость важнее удобной даты

Нельзя признать готовой аналитику, если ещё не установлено, какие операции считаются подтверждёнными. Дата не заменяет входное условие.

Правило подтверждения

Определяет допустимый факт.

→

Обмен

Передаёт этот факт.

→

Аналитика

Строит по нему показатель.

Миграция данных

Миграция включает подготовку, сопоставление, преобразование, загрузку и проверку. Успешная загрузка файла не подтверждает правильность связей и значений.

Подготовить
Очистить и сопоставить данные.

→

Перенести
Применить контролируемое преобразование.

→

Проверить
Идентичность, количество, связи и рабочий сценарий.

Поэтапное расширение

После пилота решение распространяют на новые области с учётом их отличий. Расширение должно опираться на проверенную готовность, а не только на успешную установку.

Пилот

Одна площадка и известные условия.

→

Сопоставление

Нагрузка, связь, данные и компетенции новой области.

→

Расширение

Дополнительные проверки и поддержка.

Условия переключения на новый контур

Перед переключением нужно проверить не только программу, но и незавершённые операции, данные, поддержку и порядок действий пользователей.

Данные готовы

Сопоставлены и проверены значимые записи.

Работа готова

Пройдены существенные сценарии.

Поддержка готова

Есть ответственные и процедура обращения.

Возврат к прежнему состоянию

Возврат после неудачного переключения требует понимания судьбы новых операций. Просто вернуть старую программу может быть недостаточно.

Состояние

Какие изменения уже появились после переключения.

Данные

Как сохранить или согласовать новые операции.

Решение

Кто разрешает возврат и по какому критерию.

План возврата должен учитывать данные, а не только версии приложений.

Передача в эксплуатацию

Передача результата означает, что другая сторона может продолжать работу осознанно: знает устройство, ограничения, статусы и действия при отклонении.

Материалы

Актуальные модели, версии и инструкции.

Ответственность

Владельцы сервиса, данных и поддержки.

Ограничения

Открытые вопросы и непроверенные сценарии.

Мониторинг связывается с действием

Полезный сигнал показывает не только техническую активность, но и состояние бизнес-сценария. По сигналу должно быть понятно, кто и что делает.

Сигнал

Есть неподтверждённые операции за прошедший период.

→

Значение

Суточный отчёт может быть неполным.

→

Действие

Ответственный выясняет причину и принимает решение.

Output и outcome в архитектурном проекте

Output — непосредственный результат проекта. Outcome — изменение деятельности вследствие его использования. Оба нужны, но доказываются по-разному.

Output

Развёрнут сервис и настроен обмен.

Outcome

Снизилась трудоёмкость сверки или число ошибок.

Основание

Нужны наблюдения использования, а не только акт запуска.

Виды ожидаемого эффекта

Эффект может проявляться в разных измерениях. Их нельзя складывать в одну денежную сумму без модели и оснований.

Операционный

Меньше задержек и повторной работы.

Информационный

Больше согласованности и прослеживаемости.

Экономический

Изменение денежных потоков или использования ресурсов.

Расчёт высвобождаемого времени

Учебные условия пособия: 400 операций в месяц, 6 минут обработки до изменения и 2 минуты после. Все времена относятся к одной операции.

Время = 400 × (6 − 2) = 1 600 мин ≈ 26,7 ч

400	Число операций за месяц.
6 и 2 минуты	Трудозатраты до и после изменения.
Результат	Потенциально высвобождаемый трудовой ресурс.

Разовые и регулярные затраты

Сравнение вариантов учитывает полный состав затрат, а не только лицензию или написание кода.

Разовые

Разработка, настройка, миграция и обучение.

Регулярные

Инфраструктура, сопровождение и поддержка.

Скрытые

Сверка, временный двойной ввод и дополнительные роли.

Открытое ПО не означает отсутствие затрат на внедрение и эксплуатацию.

Совокупная стоимость владения: TCO

TCO — Total Cost of Ownership. В отдельном денежном примере разовые затраты составляют 120 тыс. ₽, сопровождение — 2 тыс. ₽ в месяц; горизонт — три года.

$$\text{TCO} = 120\,000 + 36 \times 2\,000 = 192\,000\ ₽$$

Горизонт	36 месяцев.
Включено	Разовое создание и регулярное сопровождение.
Ограничение	Пример упрощён; фактический состав затрат определяется проектом.

Денежная оценка времени — не денежная экономия

Вернёмся к примеру 400 операций. При условной стоимости часа 500 ₽ высвобождаемое время имеет оценку ресурса, но не обязательно уменьшает выплаты предприятия.

Э = (1 600 / 60) × 500 ≈ 13 333 ₽/мес.

Э	Денежная оценка высвобождаемого трудового ресурса.
Условие	500 ₽ за час задано только для учебного расчёта.
Вывод	Для денежной экономии нужно отдельное решение об использовании ресурса.

Простая оценка ROI за выбранный горизонт

Учебный денежный сценарий: экономия — 12 тыс. ₽ в месяц; затраты за три года — 192 тыс. ₽. Формула в тыс. ₽, без налогов и дисконтирования.

$$ROI = (432 - 192) / 192 \times 100\% = 125\%$$

432 тыс. ₽	36 × 12 тыс. ₽ денежной экономии.
192 тыс. ₽	Все принятые затраты за тот же горизонт.
125%	Суммарная оценка за три года, не годовая доходность.

Простой срок окупаемости

В том же денежном сценарии ежемесячная экономия составляет 12 тыс. ₽, а дополнительные регулярные расходы — 2 тыс. ₽. Чистый поток равен 10 тыс. ₽.

$$T = 120\,000 / 10\,000 = 12 \text{ месяцев}$$

T	Простой срок окупаемости при постоянном потоке.
120 000 ₽	Первоначальные разовые затраты.
10 000 ₽/мес.	12 000 ₽ экономии минус 2 000 ₽ регулярных расходов.

Как избежать двойного счёта эффекта

Один результат нельзя независимо учитывать несколько раз. Сокращение времени, рост производительности и снижение затрат могут описывать одну и ту же причинную цепочку.

Проверить механизм

Что именно изменилось.

Развести показатели

Как каждый показатель связан с этим изменением.

Не суммировать без основания

Денежная оценка труда не добавляется повторно к той же экономии.

Сценарная оценка при неопределённости

Когда параметры неизвестны точно, полезно рассмотреть несколько обоснованных сценариев. Цель — показать чувствительность решения, а не замаскировать неопределённость средним числом.

Осторожный

Меньше объём пригодных операций.

Базовый

Условия принятого расчёта.

Благоприятный

Больше подтверждённый охват и меньше издержки.

План проверки эффекта после внедрения

Ожидаемый эффект проверяют на фактическом использовании. Нужно заранее определить показатель, источник данных, период и ответственного.

Показатель

Время подготовки согласованного отчёта.

Наблюдение

Измерение до и после при сопоставимых условиях.

Решение

Что делать при отсутствии ожидаемого улучшения.

Управление архитектурой продолжается после запуска

Результаты использования могут потребовать изменения правил, компонентов и плана развития. Архитектурные решения сохраняются актуальными через управляемый пересмотр.

Наблюдать

Эффект, ошибки и новые требования.

→

Оценивать

Какие исходные предположения изменились.

→

Корректировать

Обновить решение и соответствующие представления.

Учебный пример

Кейс: условия целевого изменения

В «Учебном хозяйстве» требуется согласованный суточный отчёт. Полная замена учётной системы исключена; необходимо сохранить работу площадки при временной потере связи.

Цель

Понятные статусы и единый учёт операций.

Граница

Выдача, подтверждение, обмен и сводка.

Ограничение

Учебный проект не доказывает промышленное внедрение.

Учебный пример

Кейс: существенные факты AS-IS

Сохраняем работающую регистрацию и учёт, но выделяем конкретные несогласованности.

Данные

Разные обозначения одного корма.

Обмен

Повторы без надёжной идентификации.

Ответственность

Подтверждение не всегда отделено от ввода.

Кейс: согласованное TO-BE

Целевое решение объединяет организационные и программные изменения. Новые правила должны быть реализованы в данных, интерфейсах и проверках.

Организация

Явный владелец и роль подтверждения.

Данные

Согласованный справочник и код операции.

Приложение

Контроль повтора и видимый статус обмена.

Простого списка «новая БД + API + интерфейс» здесь было бы недостаточно.

Кейс: от разрывов к инициативам

Для каждого разрыва указываем изменение и проверяемый результат.

Разрыв	Инициатива	Критерий
Разные коды	Сопоставить справочник	Операции сопоставляются
Повторный учёт	Контролировать идентификатор	Две отправки — один факт
Неясный статус	Ввести подтверждение	Неполнота отчёта видима

Кейс: дорожная карта без выдуманного календаря

Используем относительные этапы, потому что ресурсный календарь реального предприятия не задан.

Подготовка
Правила и справочник.

→

Проверка
Обмен и повторы.

→

Пилот
Одна площадка.

→

Переход
Расширение и отмена старого маршрута.

Каждый переход разрешается по критерию готовности, а не по названию этапа.

Учебный пример

Кейс: честная формулировка эффекта

По условиям учебного расчёта может высвободиться около 26,7 часа в месяц. Денежная экономия не объявляется доказанной без фактического решения об использовании времени.

Рассчитано

Потенциальный трудовой ресурс.

Нужно измерить

Действительное время и объём пригодных операций.

Нужно решить

Как использовать освобождённую возможность.

Кейс: что должна подтвердить экспертиза

Архитектурная защита показывает цепочку от задачи до проверок и явно отделяет реализованное от предполагаемого.

Связность

Проблема, требование и решение согласованы.

Реализуемость

Зависимости перехода и владельцы определены.

Границы

Остаточные риски и непроверенные условия названы.

Перенос: новый сервис согласований

Отдел хочет заменить бумажные согласования электронным сервисом. Как понять, что проект завершён, кроме факта установки программы?

Процесс

Определены роли, статусы и исключения.

Переход

Незавершённые документы не потеряны.

Эффект

Измерено изменение времени и ошибок.

Проверка понимания

Обоснуйте каждое различие коротким примером.

1. Почему TO-BE не должна быть списком новых программ?
2. Чем таблица GAP отличается от списка задач?
3. Что может и чего не может доказать прототип?
4. Почему высвобождение времени не равно денежной экономии?
5. Какие условия нужны для безопасного переключения?

Ответы к проверке

Ключевые признаки достаточного ответа:

1. TO-BE объясняет изменение деятельности и связей.
2. GAP сопоставляет состояния; задачи реализуют изменения.
3. Прототип проверяет заданную гипотезу в известных условиях.
4. Деньги зависят от решения об использовании ресурса.
5. Нужны готовые данные, проверки, поддержка и план возврата.

Итог: архитектура как обоснованное изменение

Архитектурный проект завершает не красивая схема, а объяснимая связь между задачей, устройством решения, переходом и проверкой результата.

Понять

Объект, проблему и основания.

→

Спроектировать

Требования, варианты и TO-BE.

→

Проверить и передать

Разрывы, переход, эффект и ограничения.

Эта логика станет основой практического архитектурного проекта курса.

Источники и материалы для повторения

Учебные материалы курса и первичные источники. Полные ссылки и привязка к теме находятся в заметках соответствующих слайдов.

[2] Моторин О.А., Вокуева Н.Е. Архитектура предприятия: учебно-методическое пособие. Рабочая рукопись, 2026.

[3] Моторин О.А. Лекция 1 от 01.10.2026: расшифровка выступления. Примеры и пояснения; не источник проверенных эмпирических результатов.

[27] Безопасность и ИИ-компоненты: пособие, главы 9–11. Учебные ограничения и сценарии.

[28] Kazman R., Klein M., Clements P. ATAM: Method for Architecture Evaluation. SEI, 2000.

[30] Экономические расчёты v6: авторская учебная постановка по главам 10 и 12 пособия. Параметры и ограничения приведены рядом с расчётами.